

Yes — now that I can see the **actual** `Home.jsx`, the problem is clear.

Your state is correctly initialized:

JavaScript

So the error means `getTours()` **is returning** `undefined` , and then this line eventually tries:

JavaScript

The important part is your service:

JavaScript

If `getTours()` does not return the API's array correctly, `setTours(undefined)` happens.

1. Fix `tourService.js`

Please make your `frontend/src/services/tourService.js` look like this:

```
import api from "../api";

/**
 * Get all active tours.
 *
 * The backend endpoint:
 *
 * GET /api/tours
 *
 * Expected response:
 *
 * {
 *   success: true,
 *   message: "...",
 *   data: [...]
 * }
 */
```

```

export const getTours = async () => {
  const response = await api.get("/tours");

  /**
   * Our PHP API returns the actual tours
   * inside the "data" property.
   */
  return Array.isArray(response.data?.data)
    ? response.data.data
    : [];
};

/**
 * Get a single tour by ID.
 *
 * Example:
 *
 * GET /api/tours/1
 */
export const getTour = async (id) => {
  const response = await api.get(`/tours/${id}`);

  return response.data?.data ?? null;
};

/**
 * Get prices for a specific tour.
 *
 * Example:
 *
 * GET /api/tours/1/prices
 */
export const getPrices = async (id) => {
  const response = await api.get(`/tours/${id}/prices`);

  return Array.isArray(response.data?.data)
    ? response.data.data
    : [];
};

```

JavaScript

Why this fixes it

Your PHP API response is structured approximately like:

```
{
  "success": true,
  "message": "Tours retrieved successfully.",
  "data": [
    {
      "id": 1,
      "title": "Safari Blue Zanzibar",
      "price": 150,
      "currency": "USD"
    }
  ]
}
```

JSON

Axios therefore gives you:

JavaScript

which is the entire object.

The actual tours are:

JavaScript

So `getTours()` must return:

JavaScript

not:

JavaScript

2. Make `Home.jsx` defensive

Your current `Home.jsx` is mostly correct. I recommend making one small improvement so the UI can **never crash just because the API returns an unexpected value**.

Replace your entire `Home.jsx` with this:

```
import { useEffect, useState } from "react";

import { getTours } from "../services/tourService";
import TourCard from "../components/TourCard";

const Home = () => {
  const [tours, setTours] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    const loadTours = async () => {
      try {
        setLoading(true);
        setError("");

        const tourData = await getTours();
        // ...
      } catch (error) {
        // ...
      }
    };
    loadTours();
  }, []);
};
```

3. Check `api.js`

Your `api.js` should also be configured consistently with the PHP backend.

For example:

```
import axios from "axios";

const api = axios.create({
  baseURL: "http://localhost:8000/api",
  headers: {
    "Content-Type": "application/json",
    Accept: "application/json",
  },
});

export default api;
```

Then:

calls:

```
http://localhost:8000/api/tours
```

4. Test the backend first

Before testing React, run:

Bash

You should get something similar to:

```
{
  "success": true,
  "message": "...",
  "data": [
    {
      "id": 1,
      "title": "Safari Blue Zanzibar",
      "slug": "safari-blue-zanzibar",
      "price": 150,
      "currency": "USD"
    }
  ]
}
```

JSON

```
}
```

Then restart Vite if necessary:

Bash

Open:

```
http://localhost:5173/
```

You should again see:

```
Safari Blue Zanzibar
Fumba
Full Day
From $150.00 / person
View Tour
```

One important point

The React DevTools message:

```
Download the React DevTools for a better development experience
```

is **not an error**.

The real error was:

```
Cannot read properties of undefined (reading 'length')
```

and the root cause is almost certainly the shape/value returned by `getTours()` .

Once this is fixed, **do not move on to adding more frontend features yet**. First verify that `/` loads correctly and the **View Tour** button takes you to:

```
/tours/safari-blue-zanzibar
```

Then we can build the **real commercial Tour Details page** connected to `/api/tours/{id}` and `/api/tours/{id}/prices` , followed by the normal **Book / Enquire Now** flow (without online payment).



```
const API_BASE_URL = "http://localhost:8000/api";
```

```
const api = {
  get: async (endpoint) => {
    const response = await
    fetch( `${API_BASE_URL}${endpoint}` );
```

```
const data = await response.json();
```

```
if (!response.ok || data.success === false) {
  throw new Error(
```

Show more ▾

Exactly — this explains the error.

Your `api.js` uses `fetch()`, and your `get()` method returns the **entire API response object**:

JavaScript

Your PHP API response is structured like:

```
{
  "success": true,
  "message": "Tours retrieved successfully.",
  "data": [
    {
      "id": 1,
      "title": "Safari Blue Zanzibar",
      "price": 150,
      "currency": "USD"
```

JSON

```
}
```

Therefore, if your current `tourService.js` does:

```
const response = await api.get("/tours");
```

JavaScript

then that is correct.

The problem is most likely that `tourService.js` is returning the wrong property.

Use this exact tourService.js

Replace the entire file:

frontend/src/services/tourService.js

with:

```
import api from "../api";

/**
 * Get all active tours.
 *
 * Backend:
 * GET /api/tours
 *
 * API response:
 * {
 *   success: true,
 *   message: "...",
 *   data: [...]
 * }
 */
export const getTours = async () => {
  const response = await api.get("/tours");

  return Array.isArray(response.data)
    ? response.data
    : [];
};

/**
 * Get a single tour.
 *
 * Backend:
 * GET /api/tours/{id}
 */
export const getTour = async (id) => {
  const response = await api.get(`/tours/${id}`);

  return response.data ?? null;
};
```

```
/**
 * Get all prices for a tour.
 *
 * Backend:
 * GET /api/tours/{id}/prices
 */
```

JavaScript

```
... const response = await fetch(url);

return Array.isArray(response.data)
  ? response.data
  : [];
};
```

Why?

Your `api.js` already does this:

```
const data = await response.json();
```

JavaScript

So:

JavaScript

gives you:

```
{
  success: true,
  message: "...".
```

JavaScript

Therefore:

JavaScript

is the array of tours.

Then:

JavaScript

means Home.jsx receives:

```
[
  {
    id: 1,
    title: "Safari Blue Zanzibar",
    price: 150,
    description: "1000"
```

JavaScript

-

and this works:

JavaScript

and:

JavaScript

Your api.js is fine

You **do not need to change** api.js .

Keep:

```
const API_BASE_URL = "http://localhost:8000/api";
```

```
const api = {
  get: async (endpoint) => {
    const response = await fetch(`${API_BASE_URL}${endpoint}`);

    const data = await response.json();

    if (!response.ok || data.success === false) {
      throw new Error(
        data.message || "API request failed."
      );
    }

    return data;
  },

  post: async (endpoint, body) => {
    const response = await fetch(`${API_BASE_URL}${endpoint}`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(body),
    });

    const data = await response.json();

    if (!response.ok || data.success === false) {
      throw new Error(
        data.message || "API request failed."
      );
    }

    return data;
  },

  put: async (endpoint, body) => {
    const response = await fetch(`${API_BASE_URL}${endpoint}`, {
      method: "PUT",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(body),
    });

    const data = await response.json();

    if (!response.ok || data.success === false) {
      throw new Error(
        data.message || "API request failed."
      );
    }
  }
}
```

```

    });
  }

  return data;
},

delete: async (endpoint) => {
  const response = await fetch(`${API_BASE_URL}${endpoint}`, {
    method: "DELETE",
  });

  const data = await response.json();

  if (!response.ok || data.success === false) {

JavaScript
    },
  }

  return data;
},
};

export default api;

```

Then test

Save the files and refresh:

`http://localhost:5173/`

You should get:

```

Safari Blue Zanzibar
Fumba
Full Day
From $150.00 / person
View Tour

```

If that appears, the **Home** → **API** → **tours** → **price** → **TourCard** pipeline is working correctly.

Then our next step should be the **real** `/tours/:slug` **Tour Details page**, including:

- Tour title
- Destination

- Duration
- Description
- Starting price
- Pricing information
- What's included
- What's excluded
- Activities
- Gallery/image area
- **Book / Enquire Now** CTA
- Mobile-responsive premium layout

No online payment will be added.



ok proceed

Great. Since the **Home** → **API** → **Tours** → **TourCard** flow is now established, let's proceed to the next major feature: the **real Tour Details page**.

We will build it in a way that fits the existing PHP + PostgreSQL + React architecture and **does not introduce online payment**.

Step 1 — Update the tour service

We already prepared `getTour()` and `getTourPrices()`. Make sure your `tourService.js` contains:

```
import api from "../api";

/**
 * Get all active tours.
 */
export const getTours = async () => {
  const response = await api.get("/tours");

  return Array.isArray(response.data)
    ? response.data
```

```

      : [];

};

/**
 * Get a single tour by ID.
 */
export const getTour = async (id) => {
  const response = await api.get(`/tours/${id}`);

  return response.data ?? null;
};

/**
 * Get prices for a specific tour.
 */
export const getTourPrices = async (id) => {

  JavaScript
  return Array.isArray(response.data)
    ? response.data
    : [];
};

```

Step 2 — Important issue: our URL uses the slug

Our card currently sends the visitor here:

 `/tours/safari-blue-zanzibar`   

But our backend currently has:

```
GET /api/tours/1
```

So we need to connect:

```

safari-blue-zanzibar
  ↓
find matching tour
  ↓
tour ID = 1
  ↓
GET /api/tours/1

```

↓
GET /api/tours/1/prices

For now, the cleanest solution is to load the tours and find the matching slug. Later, for a production system, we'll add:

```
GET /api/tours/slug/{slug}
```

so we don't have to download all tours just to find one.

Step 3 — Create TourDetails.jsx

Create/replace:

```
frontend/src/pages/TourDetails.jsx
```

with the complete code below:

```
import { useEffect, useState } from "react";
import { Link, useParams } from "react-router-dom";

import {
  getTours,
  getTour,
  getTourPrices,
} from "../services/tourService";

const TourDetails = () => {
  const { slug } = useParams();

  const [tour, setTour] = useState(null);
  const [prices, setPrices] = useState([]);

  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  // ...
```

Step 4 — Update App.jsx

Your routes need to include the details page.

Use:

```
import {
  BrowserRouter,
  Routes,
  Route,
} from "react-router-dom";

import Home from "../pages/Home";
import TourDetails from "../pages/TourDetails";

function App() {
  return (
    <BrowserRouter>

      <Routes>

        <Route
          path="/"
          element={<Home />}
        />

        <Route
          path="/tours/:slug"
          element={<TourDetails />}
        />

      </Routes>

    </BrowserRouter>

    JavaScript
  )
}

export default App;
```

Step 5 — Add the CSS

Now we need the page to look like a **real premium tourism product**, rather than an unstyled React page.

Open:

frontend/src/index.css

At the **bottom** of the existing CSS file, add:

```
/* =====
TOUR DETAILS PAGE
===== */

.tour-details-page {
  min-height: 100vh;
  background: #ffffff;
}

/* =====
CONTAINER
===== */

.tour-details-container {
  width: min(1180px, 92%);
  margin: 0 auto;
}

CSS
===== */

.tour-details-hero {
  position: relative;
  min-height: 560px;
  display: flex;
  align-items: flex-end;
  padding: 90px 0 70px;
  background:
    linear-gradient(
      135deg,
      #123f46,
      #0d6265 50%,
      #092f35
    );
  color: #ffffff;
  overflow: hidden;
}
```

```

.tour-details-hero::before {
  content: "";
  position: absolute;
  width: 500px;
  height: 500px;
  border-radius: 50%;
  background: rgba(255, 255, 255, 0.04);
  top: -220px;
  right: -120px;
}

.tour-details-hero::after {
  content: "";
  position: absolute;
  width: 300px;
  height: 300px;
  border-radius: 50%;
  background: rgba(255, 255, 255, 0.03);
  bottom: -180px;
  left: -80px;
}

.tour-details-hero-overlay {
  position: absolute;
  inset: 0;
  background:
    linear-gradient(
      to bottom,
      rgba(0, 0, 0, 0.08),
      rgba(0, 0, 0, 0.30)
    );
}

.tour-details-hero .tour-details-container {
  position: relative;
  z-index: 2;
}

/* =====
BREADCRUMB
===== */

.tour-details-breadcrumb {
  display: flex;
  align-items: center;
  gap: 10px;
  margin-bottom: 35px;
  font-size: 0.86rem;

```

```

        color: rgba(255, 255, 255, 0.72);
    }

    .tour-details-breadcrumb a {
        color: #ffffff;
        text-decoration: none;
    }

    .tour-details-breadcrumb a:hover {
        text-decoration: underline;
    }

    /* =====
    HERO CONTENT
    ===== */

    .tour-details-category {
        display: inline-flex;
        padding: 8px 14px;
        margin-bottom: 18px;
        border: 1px solid rgba(255, 255, 255, 0.25);
        border-radius: 999px;
        font-size: 0.75rem;
        font-weight: 700;
        letter-spacing: 0.12em;
        text-transform: uppercase;
    }

    .tour-details-hero h1 {
        max-width: 850px;
        margin: 0;
        font-size: clamp(2.6rem, 6vw, 5.2rem);
        line-height: 1.02;
        letter-spacing: -0.045em;
    }

    .tour-details-hero-content > p {
        max-width: 760px;
        margin: 25px 0 0;
        color: rgba(255, 255, 255, 0.86);
        font-size: 1.1rem;
        line-height: 1.75;
    }

    /* =====
    QUICK INFORMATION
    ===== */

```

```
.tour-details-quick-info {
  display: flex;
  flex-wrap: wrap;
  gap: 45px;
  margin-top: 42px;
}

.tour-details-quick-info div {
  display: flex;
  flex-direction: column;
  gap: 7px;
}

.tour-details-quick-info span {
  color: rgba(255, 255, 255, 0.58);
  font-size: 0.7rem;
  font-weight: 700;
  letter-spacing: 0.12em;
}

.tour-details-quick-info strong {
  font-size: 1rem;
}

/* =====
MAIN CONTENT
===== */

.tour-details-content {
  padding: 90px 0;
}

.tour-details-layout {
  display: grid;
  grid-template-columns: minmax(0, 1fr) 350px;
  gap: 70px;
  align-items: start;
}

.tour-details-main {
  min-width: 0;
}

.tour-details-section {
  padding-bottom: 70px;
  margin-bottom: 70px;
  border-bottom: 1px solid #e9e9e9;
```



```
}

.tour-details-section:last-child {
  border-bottom: 0;
  margin-bottom: 0;
}

.tour-section-label {
  display: block;
  margin-bottom: 14px;
  color: #168083;
  font-size: 0.72rem;
  font-weight: 800;
  letter-spacing: 0.14em;
}

.tour-details-section h2 {
  margin: 0 0 22px;
  color: #123f46;
  font-size: clamp(1.8rem, 3vw, 2.7rem);
  line-height: 1.15;
  letter-spacing: -0.03em;
}

.tour-details-section > p {
  max-width: 820px;
  margin: 0;
  color: #56676a;
  font-size: 1.05rem;
  line-height: 1.9;
}

/* =====
HIGHLIGHTS
===== */

.tour-highlights {
  display: grid;
  gap: 24px;
  margin-top: 35px;
}

.tour-highlight {
  display: grid;
  grid-template-columns: 52px 1fr;
  gap: 20px;
  padding: 25px;
  border: 1px solid #e6eeee;
```

```
border-radius: 18px;
background: #fbfdfd;
}

.tour-highlight-icon {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 52px;
  height: 52px;
  border-radius: 50%;
  background: #e5f4f3;
  color: #137779;
  font-size: 0.8rem;
  font-weight: 800;
}

.tour-highlight h3 {
  margin: 0 0 8px;
  color: #123f46;
  font-size: 1.05rem;
}

.tour-highlight p {
  margin: 0;
  color: #667578;
  line-height: 1.7;
}

/* =====
PRICING
===== */

.tour-pricing-table {
  margin-top: 30px;
  border: 1px solid #e5ebeb;
  border-radius: 18px;
  overflow: hidden;
}

.tour-price-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 25px;
  padding: 22px 25px;
  border-bottom: 1px solid #e8eeee;
}
```

```
.tour-price-row:last-child {
  border-bottom: 0;
}

.tour-price-row div {
  display: flex;
  flex-direction: column;
  gap: 5px;
}

.tour-price-row strong {
  color: #123f46;
}

.tour-price-row > strong {
  font-size: 1.25rem;
}

.tour-price-row span {
  color: #718083;
  font-size: 0.85rem;
}

/* =====
INFORMATION CARDS
===== */

.tour-info-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 18px;
  margin-top: 30px;
}

.tour-info-card {
  padding: 25px;
  border-radius: 16px;
  background: #f5f9f9;
}

.tour-info-card strong {
  display: block;
  margin-bottom: 10px;
  color: #123f46;
}

.tour-info-card p {
```

```
margin: 0;
color: #687779;
line-height: 1.7;
font-size: 0.92rem;
}

/* =====
SIDEBAR
===== */

.tour-details-sidebar {
  position: sticky;
  top: 30px;
}

.tour-booking-card {
  padding: 35px;
  border: 1px solid #e2e9e9;
  border-radius: 22px;
  background: #ffffff;
  box-shadow: 0 20px 60px rgba(16, 58, 62, 0.10);
}

.tour-booking-card > span {
  color: #168083;
  font-size: 0.7rem;
  font-weight: 800;
  letter-spacing: 0.13em;
}

.tour-booking-card h3 {
  margin: 14px 0 15px;
  color: #123f46;
  font-size: 1.55rem;
  line-height: 1.25;
}

.tour-booking-card > p {
  margin: 0;
  color: #687779;
  font-size: 0.94rem;
  line-height: 1.7;
}

/* =====
BOOKING PRICE
===== */
```

```

.tour-booking-price {
  display: flex;
  flex-direction: column;
  margin: 30px 0;
  padding: 22px 0;
  border-top: 1px solid #e7e7e7;
  border-bottom: 1px solid #e7e7e7;
}

.tour-booking-price small {
  color: #718083;
  margin-bottom: 4px;
}

.tour-booking-price strong {
  color: #123f46;
  font-size: 2rem;
  line-height: 1.1;
}

.tour-booking-price span {
  margin-top: 4px;
  color: #7a888a;
  font-size: 0.82rem;
}

/* =====
BOOK BUTTON
===== */

.tour-book-button {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  min-height: 54px;
  border-radius: 12px;
  background: #168083;
  color: #ffffff;
  text-decoration: none;
  font-weight: 750;
  transition:
    transform 0.2s ease,
    background 0.2s ease,
    box-shadow 0.2s ease;
}

```

```
.tour-book-button:hover {
  transform: translateY(-2px);
  background: #116b6e;
  box-shadow: 0 12px 25px rgba(22, 128, 131, 0.25);
}

/* =====
BACK LINK
===== */

.tour-back-link {
  display: block;
  margin-top: 20px;
  color: #607174;
  text-align: center;
  font-size: 0.88rem;
  text-decoration: none;
}

.tour-back-link:hover {
  color: #168083;
}

/* =====
LOADING
===== */

.tour-details-loading {
  min-height: 70vh;
  display: flex;
  align-items: center;
  justify-content: center;
}

.tour-details-loading-inner {
  text-align: center;
  color: #607174;
}

.loading-spinner {
  width: 42px;
  height: 42px;
  margin: 0 auto 18px;
  border: 3px solid #dfeaea;
  border-top-color: #168083;
  border-radius: 50%;
  animation: tourSpinner 0.8s linear infinite;
```

```
}

@keyframes tourSpinner {
  to {
    transform: rotate(360deg);
  }
}

/* =====
ERROR
===== */

.tour-details-error {
  min-height: 70vh;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 50px 0;
}

.tour-details-error-inner {
  width: min(650px, 92%);
  text-align: center;
}

.tour-details-error-inner > span {
  color: #b06d3c;
  font-size: 0.72rem;
  font-weight: 800;
  letter-spacing: 0.13em;
}

.tour-details-error-inner h1 {
  margin: 14px 0;
  color: #123f46;
  font-size: 2.4rem;
}

.tour-details-error-inner p {
  margin: 0 0 30px;
  color: #687779;
  line-height: 1.7;
}

/* =====
RESPONSIVE
===== */
```

```
@media (max-width: 900px) {  
  
  .tour-details-layout {  
    grid-template-columns: 1fr;  
    gap: 50px;  
  }  
  
  .tour-details-sidebar {  
    position: static;  
  }  
  
  .tour-booking-card {  
    max-width: 650px;  
  }  
  
}
```

```
@media (max-width: 650px) {  
  
  .tour-details-hero {  
    min-height: 520px;  
    padding: 70px 0 50px;  
  }  
  
  .tour-details-breadcrumb {  
    flex-wrap: wrap;  
    margin-bottom: 25px;  
  }  
  
  .tour-details-hero h1 {  
    font-size: 2.6rem;  
  }  
  
  .tour-details-hero-content > p {  
    font-size: 0.98rem;  
  }  
  
  .tour-details-quick-info {  
    gap: 22px;  
  }  
  
  .tour-details-content {  
    padding: 60px 0;  
  }  
  
  .tour-details-section {  
    padding-bottom: 50px;  
  }  
}
```



```
        margin-bottom: 50px;
    }

    .tour-info-grid {
        grid-template-columns: 1fr;
    }

    .tour-price-row {
        align-items: flex-start;
        padding: 20px;
    }

    .tour-booking-card {
        padding: 27px;
    }
}
```

Step 6 — Test the complete flow

Start the backend:

Bash

```
php -S localhost:8000 -t backend/public
```

Then start the frontend:

Bash

```
npm run dev
```

Go to:

```
http://localhost:5173/
```

Click:

View Tour

You should be taken to:

```
http://localhost:5173/tours/safari-blue-zanzibar
```

The page should then make these API calls:

```
GET /api/tours
  ↓
find safari-blue-zanzibar
  ↓
GET /api/tours/1
  ↓
GET /api/tours/1/prices
```

And you should see your real database information, including:

Safari Blue Zanzibar

Fumba

Full Day

From \$150.00

and the pricing information from PostgreSQL.

One thing we intentionally have NOT built yet

The button:

```
Book / Enquire Now
```

currently points to:

```
/book/safari-blue-zanzibar
```

We have not created that route yet.

That's intentional.

The next stage will be to create a proper **Booking / Enquiry system**:

```
Tour Details
  ↓
```

Book / Enquire Now



Booking Form



Name

Email

Phone

Travel Date

Number of People

Message



POST /api/bookings



PostgreSQL



Booking reference



Success confirmation

No online payment.

Before that, though, we should make one backend improvement: add the efficient endpoint:

```
GET /api/tours/slug/{slug}
```

Then the details page won't need to download **all tours** just to find one tour. That is the correct architecture for a production-quality Zan Gates Adventures system.



ChatGPT can make mistakes. Check important info.